GOOGLE MAPS

System Design & Architecture

সম্পূর্ণ Presentation Speech ও Learning Guide

বিশ্ববিদ্যালয় উপস্থাপনার জন্য তৈরি

প্রস্তুতকারী:
Mutiur Rahman (2207097)
Alok Baul (Turjo) (2207098)
Aavas Roy (2207099)
Plabon Barua (2207101)
Md. Sabbir Hossain (2207102)

বিষয়: Inside Google Maps – How a Global Mapping Platform is Designed

নোট: এটি একটি Conceptual / Reference Architecture। Google-এর proprietary internal system নয়।

ভূমিকা ও কীভাবে এই Guide ব্যবহার করবেন

এই Document-এ Google Maps System Design Presentation-এর প্রতিটি স্লাইডের জন্য বিস্তারিত ব্যাখ্যা, Presentation Speech, Technical Clarification, Teacher-এর সম্ভাব্য প্রশ্ন ও উত্তর এবং Delivery Tips দেওয়া আছে।

গুরুত্বপূর্ণ নির্দেশনা:

- প্রতিটি স্লাইডের Speech সরাসরি পড়ে প্র্যাকটিস করুন। নিজের ভাষায় পরিবর্তন করে বলতে পারবেন।
- Technical Terms ইংরেজিতেই রাখবেন, কিন্তু অর্থ বাংলায় বুঝিয়ে দেবেন।
- এটি Conceptual Architecture – Google-এর আসল internal system নয়। শিক্ষক জিজ্ঞাসা করলে এটা স্পষ্ট করে বলবেন।
- Diagram থাকলে হাতে ইঙ্গিত করে ব্যাখ্যা করবেন।
- প্রতিটি স্লাইডে ১–২ মিনিট সময় রাখবেন।

স্লাইড ১: Welcome / Title Slide

A. Slide Identification

Slide Number: 1
Slide Title: Welcome – Inside Google Maps: how a global mapping platform is designed
মূল উদ্দেশ্য: দলের পরিচয় দেওয়া এবং Presentation-এর বিষয়বস্তু পরিষ্কার করা। শিক্ষক ও সহপাঠীদের মনোযোগ আকর্ষণ করা।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

এটি Title Slide। এখানে Presentation-এর মূল বিষয় "Inside Google Maps: how a global mapping platform is designed" এবং দলের ৫ জন সদস্যের নাম ও রোল নম্বর আছে। এটি শুধু পরিচয়ের স্লাইড – কোনো technical content নেই।

C. Presentation Speech (১ মিনিট)

আসসালামু আলাইকুম / গুড মর্নিং স্যার/ ম্যাডাম এবং সহপাঠীবৃন্দ।

আজ আমরা উপস্থাপন করবো Google Maps-এর System Design ও Architecture নিয়ে। আমাদের বিষয় হলো – "Inside Google Maps: How a Global Mapping Platform is Designed" ।

আমরা ৫ জন: মুতিউর রহমান, আলোক বাউল (তুর্জো), আভাস রয়, প্লাবন বড়ুয়া এবং মোঃ সাব্বির হোসেন।

Google Maps শুধু একটি ম্যাপ অ্যাপ নয় – এটি একটি বিশাল Distributed System, যা বিশ্বজুড়ে কোটি কোটি ব্যবহারকারীকে Real-time Navigation, Traffic Intelligence এবং Location Services প্রদান করে। আজ আমরা এর Architecture, Key Technologies এবং কীভাবে এটি Scale করে সেগুলো দেখবো।

চলুন শুরু করি High-Level Architecture দিয়ে।

D. Additional Topic Knowledge

এই স্লাইডে কোনো Technical Topic নেই।

Teacher's Possible Questions

1. **প্রশ্ন: তোমরা কেন Google Maps বেছে নিলে?**

উত্তর: কারণ এটি বিশ্বের সবচেয়ে ব্যবহৃত Mapping Platform এবং এতে Geospatial Computing, Real-time Systems, Machine Learning, Distributed Systems – সবকিছুর Practical Example পাওয়া যায়।

2. **প্রশ্ন: এটি কি Google-এর আসল Architecture?**

উত্তর: না স্যার। এটি একটি Conceptual / Reference Architecture। Google-এর proprietary internal details publicly available নয়। আমরা publicly known concepts এবং research papers এর উপর ভিত্তি করে তৈরি করেছি।

Presentation Delivery Tips

- স্লাইড দেখিয়ে দলের সবাইকে সংক্ষেপে পরিচয় করিয়ে দিন।
- আত্মবিশ্বাসের সঙ্গে কথা বলুন এবং চোখে চোখ রেখে কথা বলুন।
- পরবর্তী স্লাইডে যাওয়ার আগে একটু থামুন।

স্লাইড ২: System Overview & High-Level Architecture

A. Slide Identification

Slide Number: 2
Slide Title: System Overview & High-Level Architecture
মূল উদ্দেশ্য: Google Maps-এর সামগ্রিক Architecture-এর একটা Big Picture দেওয়া – Client থেকে Data Infrastructure পর্যন্ত।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Google Maps একটি Global Geospatial Platform। এটি শুধু ম্যাপ দেখায় না – Navigation, Place Search, Real-time Traffic, Location Sharing – সব করে।

Architecture-এর Layer গুলো:

1. **Client Applications:** Web, Android, iOS – যেখান থেকে User request পাঠায়।
2. **API Gateway & Edge:** Authentication, Routing, Rate Limiting, Caching – সব প্রথমে এখানে হয়। এটি Security এবং Traffic Control-এর প্রথম স্তর।
3. **Geospatial Services:** Maps, Places, Geocoding – geographic data নিয়ে কাজ করে।
4. **Intelligence Services:** Routing, Traffic, ETA – স্মার্ট হিসাব করে।
5. **Real-Time Processing:** Location updates, Events, Incidents প্রসেস করে।
6. **Data Infrastructure:** Databases, Search, Storage – সব ডেটা এখানে থাকে।

গুরুত্বপূর্ণ নোট: এটি Conceptual Architecture। Google-এর আসল internal system publicly available নয়।

C. Presentation Speech (১.৫–২ মিনিট)

স্যার, প্রথমে আমরা Google Maps-এর High-Level Architecture দেখি।

Google Maps শুধু একটি ম্যাপ অ্যাপ নয়। এটি একটি Large-scale Distributed System, যা Geographic Data প্রসেস করে এবং কোটি কোটি User-কে Location-based Service দেয়।

উপরে দেখুন – Client Applications: Web, Android এবং iOS। User এখান থেকেই সব request করে।

Request প্রথমে যায় API Gateway ও Edge Layer-এ। এখানে Authentication, Request Routing, Rate Limiting এবং Caching হয়। অর্থাৎ কে request করছে, কতবার করছে, কোন Service-এ পাঠাবে – সব নিয়ন্ত্রণ হয় এখানে।

এর পরে মূল Service Domain গুলো:

• Geospatial Services – ম্যাপ ডেটা, রাস্তা, Place এবং Geographic Query handle করে।

• Intelligence Services – Route হিসাব, Traffic Analysis, ETA এবং Recommendation করে।

• Real-Time Processing – Location Update, Traffic Signal, Incident প্রসেস করে।

• Data Infrastructure – Database, Cache, Stream এবং Storage দেয়।

এই সব মিলে Low Latency, High Availability এবং Global Scalability নিশ্চিত করে।

মনে রাখবেন, এটি একটি Conceptual Reference Architecture – Google-এর internal proprietary system নয়।

এখন আমরা দেখবো System-এর Problem এবং Functional Requirements কী।

D. Additional Topic Knowledge

API Gateway কী?
এটি Client এবং Backend Services-এর মধ্যে Single Entry Point। Authentication, Rate Limiting, Logging, Routing – সব এখানে কেন্দ্রীভূত হয়। Microservices Architecture-এ এটি খুব গুরুত্বপূর্ণ।
কেন Layered Architecture?
প্রতিটি Layer আলাদা দায়িত্ব নেয়। ফলে একটা Layer Change করলে অন্যগুলোতে কম প্রভাব পড়ে। Scale করাও সহজ হয়।

Teacher's Possible Questions

3. **প্রশ্ন: API Gateway কেন দরকার? সরাসরি Service-এ request পাঠানো যেত না?**

উত্তর: API Gateway কেন্দ্রীয়ভাবে Authentication, Rate Limiting, Monitoring করে। প্রতিটি Service-এ আলাদা করে এগুলো implement করলে Complexity বাড়ে এবং Security Risk বাড়ে।

4. **প্রশ্ন: Geospatial Services আর Intelligence Services-এর পার্থক্য কী?**

উত্তর: Geospatial Services মূলত ডেটা store ও query করে (ম্যাপ, Place)। Intelligence Services সেই ডেটা ব্যবহার করে স্মার্ট সিদ্ধান্ত নেয় – যেমন কোন Route সবচেয়ে ভালো, ETA কত।

5. **প্রশ্ন: এটি কি Google-এর আসল Architecture?**

উত্তর: না। এটি Conceptual। Google-এর internal details secret। আমরা publicly known principles এবং research-এর উপর ভিত্তি করে ব্যাখ্যা করছি।

Presentation Delivery Tips

- Architecture Diagram-এর দিকে ইঙ্গিত করে উপর থেকে নিচে Layer ব্যাখ্যা করুন।
- "Conceptual Architecture" কথাটা জোর দিয়ে বলুন।
- প্রতিটি Layer-এর নাম বলে এক লাইনে কাজ বলুন।

স্লাইড ৩: Problem & Functional Requirements

A. Slide Identification

Slide Number: 3
Slide Title: Problem & Functional Requirements
মূল উদ্দেশ্য: System কেন দরকার এবং এটি কী কী কাজ করতে পারবে – তা পরিষ্কার করা।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Problem:
বিলিয়ন মানুষকে যেকোনো ডিভাইসে Place খুঁজে পেতে, Direction পেতে এবং Live Traffic দেখতে হয়। পৃথিবী প্রতি মুহূর্তে বদলায় – রাস্তা বন্ধ হয়, দোকান সরয়, Traffic বদলায়। উত্তর তাৎক্ষণিক এবং সর্বত্র পাওয়া যাবে এমন হতে হবে।
Functional Requirements (System কী করতে পারবে):

- ম্যাপ Display, Pan, Zoom করা
- Place ও Address Search করা
- Turn-by-turn Navigation দেওয়া
- Live Traffic ও Accurate ETA দেখানো
- Offline-এ Saved Map ব্যবহার করা
- Live Location Share করা

Users:
Drivers, Commuters, Pedestrians, Cyclists, Businesses এবং Developers (API ব্যবহারকারী)। Client: Android, iOS, Web।

C. Presentation Speech

এখন আমরা দেখি System কেন দরকার এবং এটি কী কী করতে পারবে।

Problem হলো – বিলিয়ন মানুষকে যেকোনো ডিভাইসে Place খুঁজে পেতে হয়, Direction পেতে হয়, Live Traffic দেখতে হয়। আর পৃথিবী স্থির নয়। রাস্তা বন্ধ হয়, ব্যবসা সরয়, Traffic প্রতি মিনিটে বদলায়। তাই উত্তর হতে হবে Instant এবং সর্বত্র।

Functional Requirements গুলো হলো:

ম্যাপ দেখানো, Pan-Zoom করা; Place ও Address Search; Turn-by-turn Navigation; Live Traffic ও Accurate ETA; Offline Map; এবং Live Location Sharing।

Users হলেন Drivers, Pedestrians, Cyclists, Businesses এবং API ব্যবহারকারী Developers। Client হিসেবে Android, iOS এবং Web।

এখন দেখি Non-Functional Requirements – অর্থাৎ Scale, Latency, Availability ইত্যাদি।

Teacher's Possible Questions

6. **প্রশ্ন: Functional আর Non-Functional Requirement-এর পার্থক্য কী?**

উত্তর: Functional বলে System কী করবে (Feature)। Non-Functional বলে কত ভালোভাবে করবে (Performance, Scale, Security)।

7. **প্রশ্ন: Offline Support কেন জরুরি?**

উত্তর: অনেক জায়গায় Internet দুর্বল বা নেই। Offline Map থাকলে Navigation বন্ধ হয় না।

Presentation Delivery Tips

- Problem অংশটা একটু Dramatic করে বলুন – "পৃথিবী প্রতি মুহূর্তে বদলায়"।
- Functional Requirements গুলো আঙুল তুলে গুনে গুনে বলুন।

স্লাইড ৪: Non-Functional Requirements

A. Slide Identification

Slide Number: 4
Slide Title: Non-Functional Requirements
মূল উদ্দেশ্য: System কতটা Scale করতে পারবে, কত Fast হবে, কত Reliable হবে – তা দেখানো।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

- **Scale:** Billions of users, Peak-এ Millions of requests per second।
- **Latency:** Map Tiles-এর জন্য p95 < 100ms; Route প্রায় 1 second।
- **Availability:** 99.99% uptime ≈ বছরে মাত্র ৫২ মিনিট Downtime।
- **Durability:** Map বা User Data হারানো যাবে না – Multi-region Replication।
- **Security:** Encrypted Traffic, OAuth Sign-in, Privacy Controls।

p95 Latency: মানে ৯৫% Request ১০০ms-এর কম সময়ে শেষ হয়। খুব কম Request ধীর হয়।

C. Presentation Speech

Functional Requirement বলে System কী করবে। Non-Functional বলে কত ভালোভাবে করবে।

Scale – Billions of Users, Peak-এ প্রতি সেকেন্ডে Millions of Request।

Latency – Map Tile-এর জন্য p95 Latency ১০০ মিলিসেকেন্ডের কম। অর্থাৎ ৯৫% ক্ষেত্রে Tile খুব দ্রুত আসে। Route প্রায় ১ সেকেন্ডে।

Availability ৯৯.৯৯% – বছরে মাত্র প্রায় ৫২ মিনিট Downtime সহ্য করা যায়।

Durability – কোনো Map বা User Data হারানো যাবে না। Multi-region Replication দিয়ে নিশ্চিত করা হয়।

Security – সব Traffic Encrypted, OAuth দিয়ে Sign-in, এবং User Privacy Control আছে।

এখন আমরা দেখবো Geographic Data কীভাবে Efficiently Store এবং Index করা হয়।

Teacher's Possible Questions

8. **প্রশ্ন: 99.99% Availability মানে কী?**

উত্তর: বছরে প্রায় ৫২ মিনিটের বেশি System Down থাকতে পারবে না। একে Four Nines বলে।

9. **প্রশ্ন: কেন p95 Latency বলা হয়, Average নয়?**

উত্তর: Average কিছু Slow Request-কে লুকিয়ে রাখে। p95 বলে দেয় বেশিরভাগ User-এর অভিজ্ঞতা কেমন। Tail Latency গুরুত্বপূর্ণ।

10. **প্রশ্ন: Multi-region Replication কেন?**

উত্তর: এক Region Fail হলে অন্য Region থেকে Service চালু রাখা যায় এবং Data Loss হয় না।

স্লাইড ৫: Geospatial Databases & Spatial Indexing

A. Slide Identification

Slide Number: 5
Slide Title: Geospatial Databases & Spatial Indexing
মূল উদ্দেশ্য: মিলিয়ন মিলিয়ন Geographic Object থেকে দ্রুত Nearby Place খুঁজে বের করার Technique বোঝানো।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

ম্যাপিং Platform-এ লক্ষ লক্ষ Restaurant, Road, Building থাকে। প্রতিবার সব Record Scan করলে System অচল হয়ে যাবে। তাই Spatial Indexing ব্যবহার করা হয়।

মূল Technologies:

- **PostgreSQL + PostGIS:** Geographic Object (Point, Line, Polygon) Store ও Query করার জন্য। PostGIS হলো PostgreSQL-এর Spatial Extension।
- **R-tree / GiST:** Bounding Box দিয়ে Nearby Object দ্রুত খুঁজে। GiST = Generalized Search Tree।
- **QuadTree:** একটা Area-কে ৪ ভাগে ভাগ করে Recursiveভাবে Index করে।
- **Geohash:** Latitude-Longitude-কে String-এ Encode করে Grid Cell বানায়। কাছাকাছি জায়গার Geohash Prefix একই হয়।
- **H3:** Uber-এর Hexagonal Hierarchical Grid। Traffic Aggregation-এ ভালো।
- **Redis:** Nearby Result Cache করে রাখে যাতে বারবার DB-তে না যেতে হয়।

C. Presentation Speech

Geographic Data-তে লক্ষ লক্ষ Place থাকে। প্রতিবার সব Record দেখলে Response অনেক Slow হয়ে যাবে।

তাই আমরা Geospatial Database এবং Spatial Indexing ব্যবহার করি।

PostgreSQL + PostGIS দিয়ে Geographic Object Store এবং Query করা যায়।

R-tree বা GiST Index Bounding Box ব্যবহার করে Nearby Object খুব দ্রুত বের করে।

QuadTree Area-কে চার ভাগে ভাগ করে, Geohash Coordinate-কে String Grid-এ রূপান্তর করে, H3 Hexagonal Cell ব্যবহার করে Aggregation-এর জন্য। Redis দিয়ে Result Cache করা হয়।

মূল কথা – Spatial Indexing-এর ফলে আমরা পুরো Dataset Scan না করেই Nearby Query দ্রুত করতে পারি।

এখন দেখি Routing এবং Navigation কীভাবে কাজ করে।

D. Additional Topic Knowledge – গভীর ব্যাখ্যা

1. R-tree কী?
R-tree হলো Spatial Data-এর জন্য Tree-based Index। প্রতিটি Node-এ Bounding Box থাকে। Query-এর সময় যে Box-এর সাথে Overlap নেই সেগুলো Skip করা হয়। ফলে Search Space অনেক কমে যায়।
কেন ব্যবহার হয়? Nearby Search, "আমার কাছাকাছি Restaurant" – এ ধরনের Query-তে।
উদাহরণ: ঢাকা শহরের সব Restaurant-এর মধ্যে শুধু Gulshan-এর Bounding Box-এর ভিতরেরগুলো Check করলেই হয়।
2. Geohash কী?
Geohash Latitude ও Longitude-কে Base32 String-এ Convert করে। যত লম্বা String, তত Fine Grid। কাছাকাছি দুটি Location-এর Geohash-এর Prefix একই হয়।
উদাহরণ: "u4pruyd" এবং "u4pruy" – একই Area-এর ভিতরে। Redis-এ Geohash দিয়ে Nearby Search খুব Common।
3. H3 কী?
Uber তৈরি Hexagonal Grid System। Square Grid-এর সমস্যা হলো Diagonal Distance আলাদা হয়। Hexagon-এ সব Neighbor প্রায় সমান দূরে। Traffic বা Demand Aggregation-এ খুব কাজে লাগে।
4. PostGIS কী?
PostgreSQL-এর Extension যা Geometry ও Geography Type সাপোর্ট করে। ST_DWithin, ST_Contains, ST_Distance – এ ধরনের Function দেয়।

Teacher's Possible Questions

11. **প্রশ্ন: R-tree আর QuadTree-এর পার্থক্য কী?**

উত্তর: QuadTree Area-কে Fixed ৪ ভাগে ভাগ করে (Space-driven)। R-tree Object-এর Bounding Box দিয়ে Organize করে (Data-driven)। R-tree সাধারণত Geographic Query-তে বেশি Efficient।

12. **প্রশ্ন: Geohash-এর Limitation কী?**

উত্তর: Edge-এ থাকা দুটি কাছাকাছি Point-এর Geohash Prefix আলাদা হতে পারে (Boundary Problem)। তাই কখনো কখনো Neighbor Cell-ও Check করতে হয়।

13. **প্রশ্ন: Redis কেন ব্যবহার হয় এখানে?**

উত্তর: Hot Query-এর Result Cache করে রাখলে Database Load কমে এবং Latency কমে।

স্লাইড ৬: Routing & Navigation

A. Slide Identification

Slide Number: 6
Slide Title: Routing & Navigation
মূল উদ্দেশ্য: কীভাবে Best Route বের করা হয় – Graph Algorithm + Real-time Traffic দিয়ে।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Flow:

7. **User Request:** Destination দেয়।
8. **Routing Service:** Road Network-কে Graph হিসেবে নেয় (Intersection = Node, Road = Edge)।
9. **Real-Time Data:** Traffic, Incident, Closure দিয়ে Edge Cost আপডেট করে।
10. **Optimized Route:** Fastest/Safest Path + ETA + Turn-by-turn Direction ফেরত দেয়।

Algorithms: Dijkstra, A* (এবং Production-এ Contraction Hierarchies ইত্যাদি)।

C. Presentation Speech

User যখন Destination দেয়, Request যায় Routing Service-এ।

Road Network-কে আমরা Graph হিসেবে দেখি – Intersection আর Location হলো Node, Road হলো Edge।

Dijkstra বা A Algorithm দিয়ে Efficient Path বের করা হয়। কিন্তু শুধু Shortest Path যথেষ্ট নয় – Fastest Path লাগে। তাই Real-time Traffic, Incident, Road Closure দিয়ে Edge-এর Cost আপডেট করা হয়।*

শেষে Optimized Route, ETA এবং Turn-by-turn Direction User পায়।

মূল আইডিয়া – Geographic Data + Graph Algorithm + Real-time Traffic Intelligence মিলে Modern Navigation কাজ করে।

এখন দেখি Map কীভাবে Tile আকারে Render ও Deliver করা হয়।

D. Additional Topic Knowledge

Dijkstra Algorithm:
Shortest Path বের করার Classic Algorithm। Priority Queue ব্যবহার করে ধাপে ধাপে সবচেয়ে কম Cost-এর Node Expand করে। সব Edge-এর Weight Positive হতে হয়।
A* Algorithm:
Dijkstra-এর Improved Version। Heuristic Function (যেমন Straight-line Distance to Goal) ব্যবহার করে অপ্রয়োজনীয় দিক Explore করা কমায়। তাই Practical Routing-এ A* বেশি ব্যবহৃত।
কেন শুধু Shortest Path নয়?
কারণ ১০ কিমি খালি রাস্তা হতে পারে ৩০ মিনিট, আর ১২ কিমি Traffic-মুক্ত রাস্তা হতে পারে ২০ মিনিট। তাই Cost Function-এ Time, Traffic, Road Type সব যায়।
Production-এ আরও Advanced Technique:
Contraction Hierarchies, Hub Labeling ইত্যাদি – যেগুলো Preprocessing করে Query অনেক Fast করে। Google Maps-এ এ ধরনের Technique ব্যবহার হয় বলে Research Paper-এ ইঙ্গিত আছে।

Teacher's Possible Questions

14. **প্রশ্ন: Dijkstra আর A*-এর পার্থক্য কী?**

উত্তর: Dijkstra সব দিকে সমানভাবে খোঁজে। A* Heuristic দিয়ে Goal-এর দিকে বেশি Focus করে। তাই A* সাধারণত Fast।

15. **প্রশ্ন: Real-time Traffic কীভাবে Route-এ প্রভাব ফেলে?**

উত্তর: প্রতিটি Road Segment-এর Cost (Travel Time) Dynamicভাবে আপডেট হয়। Traffic বেশি হলে Cost বাড়ে, Algorithm অন্য Path বেছে নেয়।

16. **প্রশ্ন: বড় Road Network-এ Dijkstra কি Practical?**

উত্তর: Raw Dijkstra অনেক Slow। তাই Preprocessing-based Method (Contraction Hierarchies) বা Hierarchical Routing ব্যবহার করা হয়।

স্লাইড ৭: Map Rendering & Tile Delivery

A. Slide Identification

Slide Number: 7
Slide Title: Map Rendering & Tile Delivery
মূল উদ্দেশ্য: পুরো পৃথিবীর ম্যাপ কীভাবে ছোট ছোট Tile আকারে Efficiently দেখানো হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

পুরো পৃথিবীর ম্যাপ একবারে Download করা অসম্ভব। তাই Tile-based Approach:

11. **Tile-Based:** ম্যাপকে ২৫৬×২৫৬ px-এর Square Tile-এ ভাগ করা হয়। প্রতিটি Zoom Level-এ আলাদা Tile Set।
12. **Efficient Delivery:** CDN দিয়ে বিশ্বজুড়ে Tile Serve করা হয়।
13. **Dynamic Rendering:** Marker, Label, Real-time Data Client-side যোগ হয়।
14. **Smooth Experience:** শুধু Visible Area-এর Tile Load হয়।

Raster vs Vector Tile:
Raster = Pre-rendered Image। Vector = Geometric Data যা Client-এ Style করে Render হয় (Zoom-এ Smooth, Size কম)।

C. Presentation Speech

পুরো পৃথিবীর ম্যাপ একসাথে Download করা যায় না। তাই Google Maps Tile-based Approach ব্যবহার করে।

ম্যাপকে ছোট ছোট Square Tile-এ ভাগ করা হয়– সাধারণত ২৫৬ বাই ২৫৬ পিক্সেল। User যখন Zoom বা Pan করে, শুধু Screen-এ দরকারি Tile গুলো Request করা হয়।

এই Tile গুলো CDN-এর মাধ্যমে বিশ্বজুড়ে Cache থাকে, তাই কাছের Server থেকে দ্রুত আসে।

Raster Tile হলো Ready Image, Vector Tile হলো Data যা Client নিজে Style করে। Vector-এর সুবিধা– Smooth Zoom এবং কম Data।

মূল কথা– Tile-based Delivery-এর ফলে Unnecessary Data Transfer কমে, Loading Fast হয় এবং Global Scale সম্ভব হয়।

এখন Places Search ও Discovery দেখি।

Teacher's Possible Questions

17. **প্রশ্ন: Raster আর Vector Tile-এর পার্থক্য কী?**

উত্তর: Raster Pre-drawn Image। Vector হলো Points/Lines/Polygons যা Client Render করে। Vector Zoom-এ Quality হারায় না এবং Styling Flexible।

18. **প্রশ্ন: CDN কেন এত গুরুত্বপূর্ণ?**

উত্তর: User বাংলাদেশে থাকলে Tile যদি USA থেকে আসে তাহলে Latency বেশি হয়। CDN কাছের Edge Server থেকে Serve করে।

স্লাইড ৮: Places Search & Discovery

A. Slide Identification

Slide Number: 8
Slide Title: Places Search & Discovery
মূল উদ্দেশ্য: User কীভাবে দ্রুত সঠিক Place খুঁজে পায় – Search + Ranking দিয়ে।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Pipeline:

15. **Data Source:** Places, Businesses, POIs।
16. **Search Engine:** Full-text + Geospatial Search।
17. **Ranking:** Relevance, Distance, Quality (Rating, Popularity)।
18. **Results:** Personalized ও Local।

Features: Nearby Search, Top Results, Real-time Updates, Smart Filters (Open now, Price, Type)।

C. Presentation Speech

Places Search-এ প্রথমে Place Information Database-এ থাকে। User Query এলে Search Engine Full-text এবং Geospatial Search করে।

তারপর Ranking System Distance, Relevance, Rating, Popularity দিয়ে সাজায়। শেষে Personalized Local Result দেয় – Open now, Price Range ইত্যাদি Filter সহ।

মূল আইডিয়া – Search + Location Data + Intelligent Ranking মিলে Place Discovery Fast এবং Useful হয়।

এখন Real-time Traffic দেখি।

Teacher's Possible Questions

19. **প্রশ্ন: Ranking-এ কোন Factors গুরুত্বপূর্ণ?**

উত্তর: Text Relevance, Distance from User, Rating, Number of Reviews, Popularity, এবং Personalization (User-এর আগের Preference)।

স্লাইড ৯: Real-Time Traffic & Live Updates

A. Slide Identification

Slide Number: 9
Slide Title: Real-Time Traffic & Live Updates
মূল উদ্দেশ্য: Live Traffic Data কীভাবে সংগ্রহ, প্রসেস এবং Route-এ ব্যবহার হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Flow: Live Data (Sensors, Reports, Crowdsourcing) → Traffic Analytics → Map Updates → Better Routes।
Features: Live Traffic Visualization, Incident Alerts, Smart Rerouting, Accurate ETA Updates।
Data Source: GPS Probe Data (Anonymous from Users), Road Sensors, User Reports।

C. Presentation Speech

Google Maps অবিরত Traffic Data সংগ্রহ করে – GPS Signal, Road Sensor এবং User Report থেকে।

এই Data প্রসেস করে Congestion, Accident, Closure চিহ্নিত করা হয়। তারপর Map Update হয় এবং প্রয়োজনে Alternate Route Suggest করা হয়। ETA-ও Current Condition অনুযায়ী বদলায়।

মূল কথা – Real-time Data-ই Google Maps-কে Faster এবং More Reliable Navigation দিতে সাহায্য করে।

এখন ETA Prediction-এ Machine Learning কীভাবে ব্যবহার হয় সেটা দেখি।

Teacher's Possible Questions

20. **প্রশ্ন: Crowdsourcing কীভাবে কাজ করে?**

উত্তর: অনেক User-এর Anonymous GPS Speed Data থেকে Road-এর Average Speed বের করা হয়। যত বেশি User, তত Accurate।

স্লাইড ১০: ETA Prediction & Machine Learning

A. Slide Identification

Slide Number: 10
Slide Title: ETA Prediction & Machine Learning
মূল উদ্দেশ্য: Travel Time কীভাবে ML দিয়ে Predict করা হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Data Used:

- Historical Traffic Patterns (Time of day, Day of week)
- Real-time Traffic (Speed, Congestion, Incidents)
- Contextual Factors (Weather, Events, Holidays)
- Road Features (Type, Length, Turns, Elevation)

ML Techniques: Gradient Boosting (XGBoost, LightGBM), Deep Learning (LSTM, GNN), Time Series Models।
GNN (Graph Neural Network): Road Network নিজেই Graph, তাই GNN দিয়ে Neighbor Road-এর Influence Capture করা যায়। Google-এর Research Paper-এ GNN-based ETA আছে।

C. Presentation Speech

ETA Predict করতে Google Maps Historical Pattern, Current Traffic, Weather, Event এবং Road Feature – সব ব্যবহার করে।

এই Data দিয়ে ML Model Train করা হয় – Gradient Boosting বা Deep Learning (LSTM, GNN)। Model Travel Time Predict করে এবং Condition Change হলে ETA আপডেট হয়।

মূল উদ্দেশ্য – Accurate ETA দিয়ে Navigation আরও Reliable করা।

এখন GPS Location Tracking এবং Map Matching দেখি।

D. Additional Topic Knowledge

কেন GNN?
Road Network Graph Structure। এক Road-এর Traffic প্রতিবেশী Road-কে Affect করে। GNN এই Relational Information শিখতে পারে। ২০২১ সালের Google Research Paper-এ ETA Prediction-এ GNN ব্যবহার দেখানো হয়েছে।

Teacher's Possible Questions

21. **প্রশ্ন: শুধু Historical Average দিয়ে ETA করলে কী সমস্যা?**

উত্তর: Accident বা Sudden Congestion Capture হয় না। Real-time + ML দিয়ে Dynamic Situation Handle করা যায়।

স্লাইড ১১: GPS Location Tracking & Map Matching

A. Slide Identification

Slide Number: 11
Slide Title: GPS Location Tracking & Map Matching
মূল উদ্দেশ্য: Raw GPS-এর Error কমিয়ে User-কে সঠিক Road-এ Snap করা।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Raw GPS-এ ৫–৩০ মিটার Error থাকতে পারে (Building, Tunnel, Weak Signal)।
Steps:

19. Receive GPS Signals
20. Filter & Clean (Kalman Filter ইত্যাদি)
21. Map Matching (Snap to Road) – HMM + Viterbi
22. Accurate Real-time Location

Techniques: HMM (Hidden Markov Model), Viterbi, Kalman Filter, Sensor Fusion (GPS + Wi-Fi + Cellular + IMU)।

C. Presentation Speech

Navigation-এর জন্য User-এর Exact Position জানা জরুরি। কিন্তু Raw GPS সবসময় Accurate নয় – Building বা Tunnel-এ Error হয়।

তাই প্রথমে GPS Data Filter করে Noise কমানো হয়। তারপর Map Matching Algorithm দিয়ে Estimated Position-কে সবচেয়ে সম্ভাব্য Road-এ Snap করা হয়।

HMM এবং Viterbi Algorithm এখানে Classic Technique। Sensor Fusion দিয়ে GPS-এর সঙ্গে Wi-Fi, Cellular, IMU মিলিয়ে Accuracy বাড়ানো হয়।

ফলে Navigation Smooth হয় এবং Turn-by-turn Direction Reliable হয়।

এখন Offline Maps দেখি।

D. Additional Topic Knowledge

HMM + Map Matching:
প্রতিটি GPS Observation-কে সম্ভাব্য Road Segment-এর সঙ্গে Match করা হয়। HMM-এ Hidden State = Road Segment, Observation = GPS Point। Viterbi Algorithm সবচেয়ে সম্ভাব্য Sequence of Roads বের করে।
কেন দরকার? GPS Point রাস্তার পাশে বা ভুল Lane-এ পড়তে পারে। Map Matching সেটাকে সঠিক Road-এ নিয়ে আসে।

Teacher's Possible Questions

22. **প্রশ্ন: Map Matching ছাড়া কী সমস্যা হয়?**

উত্তর: User-কে ভুল Road-এ দেখানো হতে পারে, Turn Instruction ভুল হতে পারে, ETA-ও ভুল হতে পারে।

23. **প্রশ্ন: Kalman Filter কী করে?**

উত্তর: Noisy Measurement এবং Prediction মিলিয়ে Optimal Estimate বের করে। GPS Noise কমাতে ব্যবহার হয়।

স্লাইড ১২: Offline Maps & Data Synchronization

A. Slide Identification

Slide Number: 12
Slide Title: Offline Maps & Data Synchronization
মূল উদ্দেশ্য: Internet ছাড়াই Navigation কীভাবে সম্ভব।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Steps: Download Map Area → Store Locally (Compressed) → Use Offline → Sync & Update when Online।
Technologies: On-device Storage, Data Compression (Vector Tiles), Incremental Updates, Data Synchronization।

C. Presentation Speech

User নির্দিষ্ট Area-এর ম্যাপ Download করে Device-এ Store করতে পারে। তারপর Internet ছাড়াই Search, Direction এবং Map দেখা যায়।

Online হলে Incremental Update দিয়ে শুধু পরিবর্তিত Data Sync হয়। এতে Data সাশ্রয় হয় এবং Weak Network-এও Navigation সম্ভব হয়।

এখন Location Sharing দেখি।

Teacher's Possible Questions

24. **প্রশ্ন: Offline-এ সব Feature কি কাজ করে?**

উত্তর: Basic Navigation, Search (Downloaded Area-তে), Map View কাজ করে। Real-time Traffic বা Live Update Offline-এ পাওয়া যায় না।

স্লাইড ১৩: Location Sharing & Real-Time Tracking

A. Slide Identification

Slide Number: 13
Slide Title: Location Sharing & Real-Time Tracking
মূল উদ্দেশ্য: Live Location Share করার Flow এবং Privacy Control।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Flow: User Enables Sharing → Send Live Location → Process & Update → Show on Others' Maps।
Key Points: Privacy (কে দেখবে, কতক্ষণ), Low-latency Sync, Scalable Infrastructure।

C. Presentation Speech

User Location Sharing চালু করলে Device GPS এবং অন্যান্য Source থেকে Location নিয়ে Secure ভাবে Server-এ পাঠায়।

Authorized People তাদের ম্যাপে Live Movement দেখতে পায়। User Control করতে পারে কে দেখবে এবং কতক্ষণ Sharing চালু থাকবে।

এটি Meeting Coordinate করা, Journey Track করা এবং Safety-এর জন্য কাজে লাগে।

এখন Geocoding দেখি।

Teacher's Possible Questions

25. **প্রশ্ন: Privacy কীভাবে রক্ষা করা হয়?**

উত্তর: Explicit Consent, Time-limited Sharing, Specific People-কে Access, এবং Data Encryption।

স্লাইড ১৪: Geocoding & Reverse Geocoding

A. Slide Identification

Slide Number: 14
Slide Title: Geocoding & Reverse Geocoding
মূল উদ্দেশ্য: Address ↔ Coordinate রূপান্তর কীভাবে হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Geocoding: Address → Latitude/Longitude।
Reverse Geocoding: Coordinates → Nearest Address।
Techniques: Geospatial Indexing, Text Matching & Normalization, Ranking & Disambiguation, Large-scale Place Data।

C. Presentation Speech

Geocoding মানে Human-readable Address-কে Coordinate-এ রূপান্তর। যেমন "খুলনা বিশ্ববিদ্যালয়" লিখলে তার Lat-Long বের হয়।

Reverse Geocoding উল্টোটা– ম্যাপে Click করলে Address দেখায়।

এজন্য Spatial Index, Text Normalization, Ranking লাগে– কারণ একই নামের অনেক Place থাকতে পারে।

এখন Cloud Infrastructure এবং Scalability দেখি।

Teacher's Possible Questions

26. **প্রশ্ন: Ambiguous Address কীভাবে Handle হয়?**

উত্তর: Ranking এবং Context (User Location, Previous Search) দিয়ে Best Match বেছে নেওয়া হয়। কখনো Multiple Suggestion দেখানো হয়।

স্লাইড ১৫: System Architecture & Scalability

A. Slide Identification

Slide Number: 15
Slide Title: System Architecture & Scalability
মূল উদ্দেশ্য: Cloud-based Microservices Architecture এবং Global Scale কীভাবে হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Users → Load Balancer → Microservices (Map & Tile, Places & Search, Routing & Traffic, Geocoding) → Data & Storage Layer (Spanner, Bigtable ইত্যাদি) + Cache/CDN।
Key Tech: Google Cloud, Kubernetes, Distributed Storage, Caching & CDN।

C. Presentation Speech

Google Maps Large-scale Distributed Cloud Architecture-এ চলে।

User Request Load Balancer দিয়ে যায়, তারপর বিভিন্ন Microservice-এ – Map Service, Places Service, Routing Service, Geocoding Service।

এই Services Distributed Storage (যেমন Spanner, Bigtable) ব্যবহার করে। CDN এবং Cache Latency কমায়।

Kubernetes দিয়ে Service গুলো Scale এবং Manage করা হয়। Architecture High Availability এবং Fault Tolerance-এর উপর জোর দেয়।

মূল কথা – Distributed Cloud Infrastructure-ই Global Scale-এ Reliable Performance দেয়।

এখন API Gateway এবং Microservices Communication বিস্তারিত দেখি।

D. Additional Topic Knowledge

Spanner vs Bigtable:
Bigtable – Wide-column NoSQL, Extreme Scale, High Throughput (যেমন Time-series বা Massive Key-value)। Spanner – Globally Distributed SQL Database, Strong Consistency, Relational Query। Google Maps-এর বিভিন্ন অংশে বিভিন্ন Storage ব্যবহার হতে পারে।
Kubernetes:
Container Orchestration। Service-এর Replica বাড়ানো/কমানো, Health Check, Rolling Update – সব Manage করে।

Teacher's Possible Questions

27. **প্রশ্ন: Microservices-এর সুবিধা কী?**

উত্তর: Independent Deploy, Independent Scale, Technology Flexibility, Fault Isolation।

28. **প্রশ্ন: Load Balancer কেন দরকার?**

উত্তর: Traffics Multiple Instance-এ সমানভাবে ভাগ করে এবং Healthy Instance-এ পাঠায়।

স্লাইড ১৬: API Gateway, Microservices & Communication

A. Slide Identification

Slide Number: 16
Slide Title: API Gateway, Microservices & Communication
মূল উদ্দেশ্য: Request কীভাবে Gateway দিয়ে বিভিন্ন Service-এ যায় এবং Service গুলো কীভাবে কথা বলে।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Clients (Android, iOS, Web, Third-party) → API Gateway (Routing, Auth, Rate Limiting, Monitoring) → Microservices (Map, Places, Routing, Traffic, Geocoding) → Data Layer।
Communication: REST, gRPC, Pub/Sub (Event-driven)।
Auth: OAuth 2.0, API Keys, IAM।

C. Presentation Speech

সব Client Request প্রথমে API Gateway-এ আসে। Gateway Authentication, Authorization, Rate Limiting এবং Routing করে সঠিক Microservice-এ পাঠায়।

Map Request Map Service-এ, Search Places Service-এ, Navigation Routing Service-এ যায়।

Services একে অপরের সঙ্গে REST, gRPC বা Pub/Sub দিয়ে কথা বলে। এতে প্রতিটি Service স্বাধীনভাবে Develop, Deploy এবং Scale করা যায়।

এখন Caching এবং CDN দেখি।

Teacher's Possible Questions

29. **প্রশ্ন: gRPC কেন ব্যবহার হয়?**

উত্তর: Binary Protocol, High Performance, Strong Typing (Protobuf)। Internal Service-to-Service Communication-এ REST-এর চেয়ে Efficient।

30. **প্রশ্ন: Rate Limiting কেন?**

উত্তর: Abuse এবং DDoS থেকে বাঁচাতে, এবং Fair Usage নিশ্চিত করতে।

স্লাইড ১৭: Caching Architecture & CDN Optimization

A. Slide Identification

Slide Number: 17
Slide Title: Caching Architecture & CDN Optimization
মূল উদ্দেশ্য: Latency এবং Backend Load কমাতে Caching ও CDN কীভাবে ব্যবহার হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Layers: Users → CDN (Edge) → Caching Layer (In-memory + Distributed) → Backend Services।
Strategies: Tile Caching, Data Caching (Places, Results), Cache Policies (TTL, Eviction, Refresh)।
Tech: Google Cloud CDN, Redis/Memcached, Distributed Cache, Cache Invalidation।

C. Presentation Speech

Map Tile এবং Frequent Data বারবার Backend থেকে আনলে Latency এবং Cost বাড়ে।

তাই CDN Edge Server-এ Tile Cache করে রাখে – User-এর কাছের Server থেকে Serve হয়।

আরও In-memory Cache (Redis) এবং Distributed Cache ব্যবহার হয়। TTL এবং Invalidation দিয়ে Data Fresh রাখা হয়।

ফলে Latency কমে, Database Load কমে এবং User Experience ভালো হয়।

এখন Security এবং Privacy দেখি।

Teacher's Possible Questions

31. **প্রশ্ন: Cache Invalidation কঠিন কেন?**

উত্তর: কখন Data পুরনো হয়েছে তা জানা এবং সব Edge/Cache থেকে সঠিক সময়ে মুছে ফেলা Challenging। "There are only two hard things in Computer Science: cache invalidation and naming things."

32. **প্রশ্ন: সব কিছু কি Cache করা যায়?**

উত্তর: না। Highly Dynamic Data (Live Location, Real-time Traffic) কম সময়ের জন্য বা আলাদাভাবে Handle করতে হয়।

স্লাইড ১৮: Security, Authentication & Data Privacy

A. Slide Identification

Slide Number: 18
Slide Title: Security, Authentication & Data Privacy
মূল উদ্দেশ্য: User Data এবং Platform কীভাবে Secure রাখা হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Flow: User → Authentication (Google Account, OAuth 2.0) → Secure Access (HTTPS/TLS, API Gateway) → Services।
Mechanisms: AuthN/AuthZ, Encryption in Transit & at Rest, Privacy Controls, Access Control, Rate Limiting, Fraud Detection।
Principles: Data Minimization, User Consent, Transparency, Regulatory Compliance (GDPR ইত্যাদি)।

C. Presentation Speech

Google Maps Sensitive Data handle করে– Location History, Travel Pattern। তাই Security খুব গুরুত্বপূর্ণ।

OAuth 2.0 এবং Google Account দিয়ে Authentication হয়। সব Communication TLS/HTTPS দিয়ে Encrypted। Stored Data-ও Encrypt করা হয়।

User Privacy Control দিয়ে Location History Manage করতে পারে, Data Delete করতে পারে। Rate Limiting এবং Abuse Detection দিয়ে Platform Protect করা হয়।

মূল কথা– Strong Authentication, Encryption এবং Privacy Control দিয়ে Trust বজায় রাখা হয়।

এখন Bottlenecks, Failures এবং Trade-offs দেখি।

Teacher's Possible Questions

33. **প্রশ্ন: OAuth 2.0 কী?**

উত্তর: Authorization Framework। User Password না দিয়েই Third-party App-কে Limited Access দেওয়া যায় (Token-based)।

34. **প্রশ্ন: Encryption in Transit আর at Rest-এর পার্থক্য?**

উত্তর: In Transit = Data নেটওয়ার্কে যাওয়ার সময় (TLS)। At Rest = Disk-এ Stored থাকা অবস্থায় Encryption।

স্লাইড ১৯: Bottlenecks, Failures & Trade-offs

A. Slide Identification

Slide Number: 19
Slide Title: Bottlenecks, Failures & Trade-offs
মূল উদ্দেশ্য: System কোথায় Strain হয় এবং কী কী Trade-off করতে হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Bottlenecks:
- Hot Tiles (ব্যস্ত শহরে Rush Hour-এ একই Tile-এর অনেক Request)
- খুব বড় Road Graph-এ Route Search
- Live Location Update-এর Fan-out

Failures & Responses:
- Region Outage → Failover to another Region
- Slow Service → Timeout, Retry, Circuit Breaker
- Bad Data Release → Staged Rollout + Rollback

Trade-offs:
- Freshness vs Cost (বেশি Cache করলে পুরনো Data দেখাতে হতে পারে)
- Accuracy vs Speed (Instant Answer-এর জন্য Approximate Route)
- Consistency vs Availability (Map Read-এ Availability প্রাধান্য)

C. Presentation Speech

কোনো System-ই Perfect নয়। Google Maps-এও Bottleneck আছে – ব্যস্ত শহরের Hot Tile, বিশাল Graph-এ Routing, Live Location-এর বিপুল Update।

Failure Handle করতে Region Failover, Circuit Breaker, Staged Rollout ব্যবহার হয়।

Trade-off-ও আছে – Data কতটা Fresh রাখবেন বনাম Cost, Accuracy বনাম Speed, Consistency বনাম Availability।

System Design-এ এই Trade-off গুলো সচেতনভাবে বেছে নিতে হয়।

এখন Data Pipeline দেখি – ম্যাপ কীভাবে আপডেট হয়।

Teacher's Possible Questions

35. **প্রশ্ন: Circuit Breaker কী?**

উত্তর: কোনো Service বারবার Fail করলে কিছুক্ষণের জন্য Request পাঠানো বন্ধ রাখা হয় যাতে System আরও খারাপ না হয় এবং Recover করার সুযোগ পায়।

36. **প্রশ্ন: CAP Theorem-এর সঙ্গে কোন Trade-off সম্পর্কিত?**

উত্তর: Consistency vs Availability। Map Read-এ সাধারণত Availability বেছে নেওয়া হয় (AP System)।

স্লাইড ২০: Maps Data Pipeline & Data Processing

A. Slide Identification

Slide Number: 20
Slide Title: Maps Data Pipeline & Data Processing
মূল উদ্দেশ্য: Real-world Data কীভাবে Collect, Process এবং Map-এ Update হয়।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Pipeline: Data Sources → Ingestion → Processing (at scale) → Validation & Enrichment → Map Data Update।
Sources: Satellite & Aerial Imagery, Street View, User Contributions, Traffic & Sensor Data, Authoritative Data (Government/Partners)।
Processing: Image Analysis (ML), Data Fusion, Change Detection, Verification (Automated + Human), Incremental Update।

C. Presentation Speech

Google Maps Static নয়– এটি Continuously Updated।

Data আসে Satellite, Street View, User Edit, GPS, Government Source থেকে। Ingestion-এর পর Large-scale Processing হয়– ML দিয়ে Road, Building Detect করা, Change Detection, তারপর Validation।

Verified Change গুলো Incremental ভাবে Map Database-এ যায়।

মূল কথা– Google Maps একটি Continuously Updated Geographic Information System, যা Large-scale Data Processing এবং AI দিয়ে চলে।

এখন AI এবং Computer Vision-এর ভূমিকা দেখি।

Teacher's Possible Questions

37. **প্রশ্ন: User Contribution কীভাবে Verify হয়?**

উত্তর: Automated Check + অন্যান্য Source-এর সঙ্গে Consistency + কখনো Human Reviewer।

স্লাইড ২১: AI, Computer Vision & Satellite Imagery

A. Slide Identification

Slide Number: 21
Slide Title: AI, Computer Vision & Satellite Imagery
মূল উদ্দেশ্য: AI কীভাবে Imagery থেকে Geographic Feature বের করে Map আপডেট করে।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Flow: Image Collection (Satellite, Aerial, Street View) → AI Analysis (CV + ML) → Feature Extraction (Roads, Buildings, Places) → Validation → Map Update।
Examples: Road Detection, Building Detection, Place Recognition, Change Detection।

C. Presentation Speech

Satellite, Aerial এবং Street View Image থেকে AI এবং Computer Vision দিয়ে Road, Building, Business, Landmark Detect করা হয়।

Change Detection দিয়ে নতুন Construction বা পরিবর্তন ধরা হয়। তারপর Validation করে Map Update করা হয়।

এভাবে Massive Visual Data Structured Geographic Data-তে রূপান্তরিত হয়।

এখন Key Lessons এবং References দেখি।

Teacher's Possible Questions

38. **প্রশ্ন: শুধু AI দিয়েই কি Map Update হয়?**

উত্তর: না। AI Candidate বের করে, কিন্তু Critical Change-এ Human Review এবং অন্যান্য Source-এর সঙ্গে Cross-check হয়।

স্লাইড ২২: Key Lessons & Sources

A. Slide Identification

Slide Number: 22
Slide Title: Key Lessons & Sources
মূল উদ্দেশ্য: মূল শিক্ষাগুলো সারাংশ করা এবং Reference দেওয়া।

B. সহজ বাংলায় বিস্তারিত ব্যাখ্যা

Key Lessons (স্লাইডে):

23. Cache first: বেশিরভাগ Request Service পর্যন্ত না গিয়ে Cache থেকেই মিটানো উচিত।
24. Split into small services that scale independently।
25. Design for failure in every region।
26. Treat data freshness as a product choice with a cost।

Important References:

- ETA Prediction with Graph Neural Networks in Google Maps (Derrow-Pinion et al., 2021)
- Hidden Markov Map Matching (Newson & Krumm, 2009)
- Contraction Hierarchies (Geisberger et al.)
- A* Algorithm (Hart et al., 1968)
- Bigtable, Spanner Papers
- The Tail at Scale (Dean & Barroso)

C. Presentation Speech

শেষে কিছু Key Lesson:

১. Cache First – বেশিরভাগ Request কখনো Backend Service-এ না গেলেই ভালো।

২. ছোট ছোট Independent Service বানান যেগুলো আলাদা Scale করা যায়।

৩. প্রতিটি Region-এ Failure ধরে নিয়ে Design করুন।

৪. Data Freshness একটি Product Decision – এর Cost আছে।

আমরা কিছু Important Research Paper-ও Reference করেছি– GNN-based ETA, HMM Map Matching, Contraction Hierarchies, Bigtable, Spanner ইত্যাদি।

এখন শেষ Scride – Thank You।

স্লাইড ২৩: Thank You

A. Slide Identification

Slide Number: 23
Slide Title: Thank You
মূল উদ্দেশ্য: Presentation শেষ করা এবং প্রশ্নের জন্য উন্মুক্ত করা।

C. Presentation Speech

স্যার/ম্যাডাম এবং সহপাঠীবৃন্দ, আমাদের Presentation এখানেই শেষ।

আমরা দেখলাম Google Maps শুধু একটি ম্যাপ অ্যাপ নয় – এটি Geospatial Engineering, AI, Real-time Processing এবং Cloud Computing-এর এক বিশাল সংমিশ্রণ।

ধন্যবাদ সবাইকে মনোযোগ দিয়ে শোনার জন্য। আপনাদের কোনো প্রশ্ন থাকলে আমরা উত্তর দিতে প্রস্তুত।

Final Preparation Section

১. পুরো Presentation-এর ৫ মিনিটের বাংলা Summary

Google Maps একটি Global Geospatial Platform যা Map Visualization, Navigation, Place Discovery এবং Real-time Traffic Intelligence দেয়। এটি Client Application থেকে শুরু করে API Gateway, বিভিন্ন Microservice (Map, Places, Routing, Geocoding), Real-time Processing এবং Distributed Data Infrastructure নিয়ে গঠিত।

Geographic Data Spatial Indexing (R-tree, Geohash, H3) দিয়ে Efficiently Query হয়। Routing-এ Graph Algorithm (A*, Dijkstra) + Real-time Traffic ব্যবহার হয়। Map Tile-based Delivery এবং CDN দিয়ে Global Performance নিশ্চিত হয়। ETA-তে Machine Learning (GNN সহ) ব্যবহার হয়। GPS Error Map Matching (HMM) দিয়ে ঠিক করা হয়।

System Microservices + Kubernetes + Distributed Storage (Spanner/Bigtable) + Aggressive Caching-এ চলে। Security-এ OAuth, TLS, Privacy Control আছে। Data Pipeline Satellite, Street View এবং User Contribution থেকে AI দিয়ে Map আপডেট করে।

মূল শিক্ষা: Cache first, Independent scalable services, Design for failure, এবং Data freshness একটি Costly Product Choice। এটি Conceptual Architecture – Google-এর proprietary internal system নয়।

২. ১০ মিনিটের Presentation Script (সংক্ষিপ্ত সংস্করণ)

শুভ সকাল। আজ আমরা Google Maps-এর System Design নিয়ে কথা বলব।

Google Maps একটি Distributed Geospatial Platform। Client থেকে Request API Gateway দিয়ে বিভিন্ন Service-এ যায়– Geospatial, Intelligence, Real-time এবং Data Layer।

Functional Requirement: Map, Search, Navigation, Live Traffic, Offline, Location Sharing। Non-functional: Billions of users, <100ms tile latency, 99.99% availability।

Spatial Indexing ছাড়া Nearby Search অসম্ভব। PostGIS, R-tree, Geohash, H3, Redis – এগুলো কাজ করে। Routing-এ Graph + Real-time Traffic। Map Tile + CDN দিয়ে Delivery।

ETA-তে ML, Location-এ Map Matching, Offline-এ Local Storage + Sync। Architecture Microservices + Cloud + Cache। Security OAuth + Encryption + Privacy।

Bottleneck আছে, Trade-off আছে। Data Pipeline AI দিয়ে Continuous Update করে। মূল কথা– এটি শুধু ম্যাপ নয়, একটি Intelligent Distributed System।

ধন্যবাদ। প্রশ্ন থাকলে বলুন।

৩. ৩০টি সম্ভাব্য Viva Question ও Answer (নমুনা)

27. **Google Maps কি শুধু একটি অ্যাপ?** না, এটি Large-scale Distributed Geospatial System।
28. **API Gateway-এর কাজ কী?** Auth, Rate Limiting, Routing, Monitoring।
29. **কেন Spatial Index দরকার?** মিলিয়ন Record Scan না করে Nearby Query Fast করার জন্য।
30. **R-tree কী?** Bounding Box-based Spatial Index।
31. **Geohash-এর সমস্যা কী?** Boundary-তে কাছাকাছি Point-এর Hash আলাদা হতে পারে।
32. **A* কেন Dijkstra-এর চেয়ে ভালো?** Heuristic দিয়ে Goal-oriented Search করে।
33. **Map Matching কেন দরকার?** Raw GPS Error থাকে, Road-এ Snap করতে হয়।
34. **HMM Map Matching-এ কীভাবে ব্যবহার হয়?** Road Segment = Hidden State, GPS = Observation, Viterbi দিয়ে Best Path।
35. **Raster vs Vector Tile?** Raster = Image, Vector = Data + Client Rendering।
36. **CDN-এর সুবিধা?** Low Latency, Reduced Origin Load।
37. **ETA-তে GNN কেন?** Road Network Graph, Neighbor Influence Capture করে।
38. **Microservices-এর সুবিধা?** Independent Scale, Deploy, Fault Isolation।
39. **99.99% Availability মানে?** বছরে ≈৫২ মিনিট Downtime।
40. **Cache Invalidation কঠিন কেন?** Freshness এবং Consistency Maintain করা কঠিন।
41. **OAuth 2.0 কী?** Token-based Authorization Framework।
42. **Circuit Breaker কী?** Failing Service-এ Request বন্ধ রেখে System Protect করা।
43. **এটি কি Google-এর আসল Architecture?** না, Conceptual/Reference Architecture।

44. **Hot Tile Problem কী?** একই Tile-এর অতিরিক্ত Request-এ Bottleneck।
45. **Data Freshness vs Cost?** বেশি Fresh রাখতে বেশি Compute/Network Cost।
46. **Sensor Fusion কী?** GPS + Wi-Fi + Cellular + IMU মিলিয়ে Accuracy বাড়ানো।
47. **PostGIS কী?** PostgreSQL-এর Spatial Extension।
48. **H3 কেন Hexagon?** সব Neighbor প্রায় সমান দূরত্ব, Aggregation-এ ভালো।
49. **Spanner-এর বৈশিষ্ট্য?** Globally Distributed, Strong Consistency, SQL।
50. **Offline Map কীভাবে কাজ করে?** Area Download → Local Store → Offline Use → Online Sync।
51. **Change Detection কী?** Imagery থেকে নতুন/পরিবর্তিত Feature খুঁজে বের করা।
52. **Rate Limiting কেন?** Abuse Prevention এবং Fair Usage।
53. **p95 Latency কেন গুরুত্বপূর্ণ?** Tail Latency User Experience নির্ধারণ করে।
54. **Fan-out Problem কী?** একটি Update অনেক Client-এ পাঠাতে Resource বেশি লাগে।
55. **কেন Design for Failure?** Hardware/Network Fail অনিবার্য, System Survive করতে হবে।
56. **মূল Takeaway কী?** Google Maps = Geospatial + AI + Real-time + Cloud-এর Intelligent Distributed System।

৪. গুরুত্বপূর্ণ ২০টি Technical Term-এর বাংলা অর্থ

57. **Geospatial:** ভূ-স্থানিক / Geographic Data সংক্রান্ত
58. **Spatial Indexing:** স্থানিক তথ্য দ্রুত খোঁজার Index
59. **API Gateway:** সব API Request-এর প্রবেশদ্বার
60. **Microservices:** ছোট ছোট স্বাধীন Service
61. **CDN:** Content Delivery Network – কাছের Server থেকে Content
62. **Latency:** বিলম্ব / Response Time
63. **Throughput:** প্রতি সেকেন্ডে কত Request Handle করতে পারে
64. **Map Matching:** GPS-কে রাস্তার সঙ্গে মিলিয়ে নেওয়া
65. **ETA:** Estimated Time of Arrival
66. **Geocoding:** Address → Coordinate
67. **Reverse Geocoding:** Coordinate → Address
68. **Tile:** ম্যাপের ছোট ছোট Square অংশ
69. **Heuristic:** আনুমানিক দূরত্ব/খরচ যা Search Guide করে
70. **Circuit Breaker:** Failing Service থেকে সাময়িকভাবে বিচ্ছিন্ন করা
71. **Fan-out:** একটি Event অনেক জায়গায় পাঠানো
72. **TTL:** Time To Live – Cache কতক্ষণ থাকবে
73. **OAuth:** Secure Authorization Framework
74. **Replication:** ডেটার একাধিক কপি বিভিন্ন জায়গায়
75. **Failover:** এক System Fail হলে অন্যটিতে চলে যাওয়া
76. **Idempotent:** একই Operation বারবার করলেও একই ফল

৫. System Architecture-এর End-to-End Working Process

1. User Mobile/Web App-এ Destination বা Place Search করে।

2. Request API Gateway-এ যায় → Auth Check, Rate Limit, Route to Service।

3. Places/Geocoding Service Coordinate বের করে। Routing Service Graph + Traffic দিয়ে Best Route বের করে।

4. Map Tile CDN/Cache থেকে আসে। ETA ML Model দিয়ে Predict হয়।

5. User-এর GPS Map Matching দিয়ে Road-এ Snap হয়। Live Traffic Update হয়।

6. প্রয়োজনে Reroute হয়। Offline থাকলে Local Data ব্যবহার হয়।

7. পেছনে Continuous Data Pipeline Imagery ও Sensor Data দিয়ে Map আপডেট করে।

৬. "Explain the overall architecture of Google Maps" – সুন্দর উত্তর

স্যার, Google Maps-এর Overall Architecture একটি Conceptual Distributed System হিসেবে বর্ণনা করা যায়।

সবচেয়ে উপরে Client Applications – Android, iOS, Web। Request প্রথমে API Gateway এবং Edge Layer দিয়ে যায়, যেখানে Authentication, Rate Limiting এবং Routing হয়।

এরপর Microservices Layer – Map/Tile Service, Places Service, Routing & Traffic Service, Geocoding Service ইত্যাদি। এরা Independent ভাবে Scale হয়।

Data Layer-এ Distributed Storage (যেমন Spanner, Bigtable), Spatial Database, Cache এবং CDN থাকে। Real-time Pipeline Location এবং Traffic Update প্রসেস করে।

AI/ML ETA Prediction এবং Imagery থেকে Feature Extraction-এ সাহায্য করে। পুরো System Multi-region, Fault-tolerant এবং Heavily Cached।

এটি Google-এর proprietary internal architecture নয় – এটি publicly known principles-এর উপর ভিত্তি করে তৈরি Conceptual Reference Architecture।

৭. Presentation-এর শেষে Conclusion Speech

পরিশেষে বলতে চাই – Google Maps আমাদের দেখায় কীভাবে Geospatial Data, Graph Algorithm, Machine Learning, Real-time Systems এবং Cloud Infrastructure একসাথে মিলে একটি Global Scale Service তৈরি করে।

আমরা শিখেছি Cache-এর গুরুত্ব, Independent Service-এর সুবিধা, Failure-কে Design-এর অংশ করা এবং Data Freshness-এর Cost।

এই Architecture Conceptual হলেও System Design-এর মূল নীতিগুলো শেখার জন্য অত্যন্ত মূল্যবান।

আমাদের Presentation মনোযোগ দিয়ে শোনার জন্য সবাইকে আন্তরিক ধন্যবাদ। প্রশ্ন থাকলে আমরা আনন্দের সঙ্গে উত্তর দেব।

— শুভকামনা —

ভালো করে প্র্যাকটিস করুন। আত্মবিশ্বাসের সঙ্গে উপস্থাপন করুন।